

Design of Remote Monitoring System for Ventilator Based on Golang and MongoDB

Jingbo Hu

School of Mechanical Engineering
Southwest Jiaotong University
Chengdu, China
e-mail: hujingbo1998@foxmail.com

Yanrong Zhang

School of Mechanical Engineering
Southwest Jiaotong University
Chengdu, China
e-mail: yrzhang@home.swjtu.edu.cn

Abstract—In order to solve the contradiction between the large number of ventilators, large amount of data and limited medical personnel, this paper proposes a solution based on Golang and MongoDB. In this scheme, the ventilator uploads the device data information in real time through UDP. Based on Golang concurrency and EPOLL-ET mode, the server realizes high performance and scalability. MongoDB is used to store structured data. Compared with traditional solutions, this scheme has a more convenient way of development and stronger horizontal expansion ability, which can expand to more medical device. The test results show that the single Web service supports the maximum QPS of 105485 for short connections and 334687 for long connections. The single device service can support 3000 online devices and consumes fewer system resources.

Keywords—ventilator, server, MongoDB, multithreading, Golang

I. INTRODUCTION

Ventilator is a kind of instrument that can effectively replace, control and assist people's normal breathing. In clinic, ventilator is usually used in surgery, first aid and treatment of respiratory diseases. With the continuous improvement of people's life quality, ventilator is also increasingly used in daily life, such as to improve breathing conditions during sleep.

Generally, ventilators need to collect a large number of patients' real time data and device status data of the ventilator itself, such as the patient's respiratory pressure and respiratory frequency, and the working status and mode of the ventilator device. However, most ventilators on the market now usually use screens or physical buttons for interaction, so the efficiency of data acquisition and control of ventilators is low. According to research, the more patients the medical staff care for on average, the more intense psychological fatigue[1]. Therefore, when processing the data of multiple ventilators at the same time, medical staff will usually experience a decrease in response speed. And operation errors may occur, resulting more serious medical accidents[2].

With the continuous development of the Internet+ field, people can combine traditional communication device with the Internet, centralize data display of different devices, and realize remote control and other operations. On the one hand, it can improve the work efficiency of medical staff, but also reduce the contact between medical staff and patients, and reduce the risk of infectious disease contact. At the same time, the

probability of doctor-patient conflict is reduced. On the other hand, due to the support of Internet-related technologies, intelligent control of device can be realized remotely[3-5], and data can be preprocessed[6], such as classifying alarm information and informing relevant personnel according to the priority. In this project, the control chip used by the ventilator equipment is dual-core STM32, and the UDP communication interface is realized using LwIP protocol.

Therefore, it is necessary to develop a set of Internet of Things system with strong scalability and security from the end of medical device to the front-end control end to realize centralized remote monitoring of devices.

The solution of traditional services is mostly Java+MySQL framework. Following the continuous development of Java, the Java specification becomes increasingly burdened, correspondingly leading to the more complex and large Java framework. Therefore, the service program in this system is written by the more concise C++ and Golang. As a relational database, MySQL similarly has some defects, such as low flexibility of design constraints, poor scalability. Furthermore, it can only use standard language interaction. The design and interaction methods of NoSQL are more flexible, and users can also choose different NoSQL according to the characteristics of stored data[7]. In this project, MongoDB is used to store structured data, which makes the system more scalable.

II. SYSTEM ARCHITECTURE

The system needs to collect the status of multiple ventilators, display the data on the Web side, and can remotely control the ventilators through the Web page. At the same time, the system needs to persist the collected data for subsequent analysis.

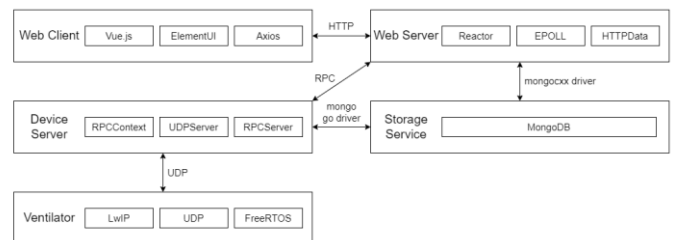


Fig. 1. System architecture.

The system uses B/S architecture as the core architecture and consists of five small systems, which are Web client, Web

server, device server, storage server and ventilator device, as shown in Fig. 1.

As the terminal part directly used by users, the Web client uses Vue.js as the system framework to control the data on the page. Part of the page UI uses some components of the ElementUI library, and the network communication part uses Axios network request library to partially update the data on the page.

The Web server is written in C++, and the basic components are implemented by referring to muduo network library. The Web client sends the request to the Web server through the HTTP protocol, and the system calls the HTTPData module to parse the HTTP packet, and calls the corresponding write function to return the corresponding data according to different requests, or wraps the request as an RPC request through the RPC module and forwards it to the device server, and listens to the return data.

The device server was written in Golang, and the pluggable design of RPC protocol, UDP protocol and storage service was implemented based on the interface feature of Golang. The Web server sent the user's control request to the device server through the RPC (Remote Procedure Call) protocol, and the device uploaded and received information using the UDP protocol.

The background storage service adopts the replica set deployment mode of MongoDB to store structured data, including device information, device data and user data. The system uses the MongoDB official driver to add, delete, check and change the database in the service program, and encapsulates the corresponding interface in the two service programs.

III. DEVICE SERVICE PROGRAM DESIGN

A. Communication Protocol Design of device server

Considering that the device side of the ventilator uses an embedded microprocessor as the main control chip and needs to serve the control of the peripheral of the ventilator itself, the communication between the ventilator and the server side of the device should minimize the occupation of CPU resources. At present, the mainstream network transport layer protocols are TCP and UDP. Compared with TCP, UDP has a faster speed and occupies fewer system resources[8]. Therefore, this system uses UDP as the communication protocol between the ventilator and the device server.

Through the requirement analysis of the monitoring task, the communication between the device and the device server is divided into four parts: the device uploading real-time data to the device server, sending heartbeat to confirm its survival, and the device server sending control instructions to the device and receiving feedback control results. In order to simplify the design and reduce unnecessary communication, real-time data, heartbeat information and feedback control result information are merged into one information in this paper, that is, real-time data and feedback results are included in heartbeat information.

B. Device Service Program Design

At present, the device management system is mostly implemented by the three frameworks of Java EE[9-10]. Since

Java appeared in 1995, the Java specification has been constantly updated. With the continuous emergence of new features, Java has begun to become more and more bloated in order to be compatible with the old version. Golang is an open source programming language introduced by Google in 2009. It has a concise syntax and strong security while guaranteeing performance. Compared with the traditional Java framework, the program written in Golang is more concise and easy to read. At the same time, Golang's powerful concurrency and network features make Golang more suitable for network background, such as goroutine for user-mode coroutine. Compared with the threading features of other languages, the overhead of system memory and startup and destruction is less, and tens of thousands of goroutines can be started easily, which makes it perform better in concurrent environments.

The main thread runs a UDPServer instance to realize the UDP service to the device. The RPC receiver thread runs an instance of RPCServer to implement the service of control information.

a) *Main thread*: The process of the main thread is shown in Fig. 2. Firstly, the main thread opens the UDP port and continuously accesses the UDP buffer to query whether there is a new packet. When a new packet enters the buffer, the main thread will take out the packet and open a goroutine to process the packet. The dashed box in Fig. 2 shows the workflow of the goroutine. Firstly, the values of each field in the message are parsed, and then the ExpiredHandler interface and DataBase interface in UDPServer are called to update the online table and the address table, and update the data in the storage service respectively. The feedback is required to be sent to the appropriate channel.

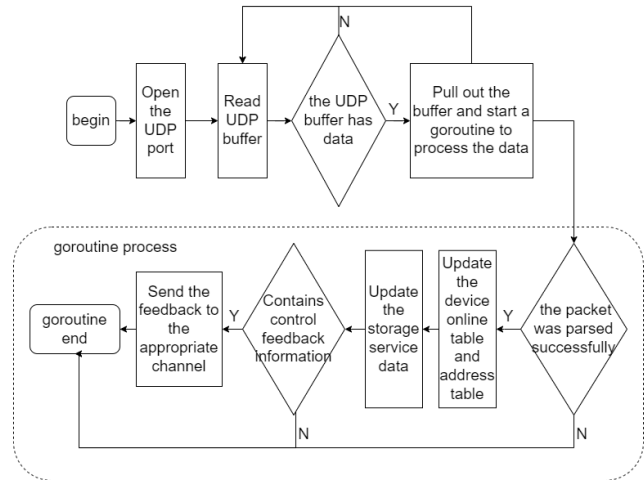


Fig. 2. Main thread process in device service program.

Considering that the device server needs to handle the forwarding of RPC packets, that is, the control request of the ventilator, the device server needs to update the device online table and address table in the local memory before updating the device status data in the storage service. The ExpiredHandler interface included in the UDPServer instance is used to update the online and address tables. It contains two methods, addTimerNode() and handleExpired(), which add a device time node to the data structure. The latter is used to remove expired device time nodes. The ExpiredHandler of this paper is

implemented based on queue and hash table. The detailed workflow is shown in Fig. 3 and Fig. 4. The module maintains an online queue and the number of nodes of each device in the queue, and updates the online status of the device by using the FIFO feature of the queue and the delayed deletion technique. The advantage of this architecture is that both `addTimerNode()` and `handleExpired()` are $O(1)$ amortized time complexity.

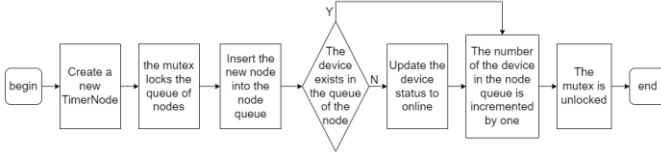


Fig. 3. process in `addTimerNode()`.

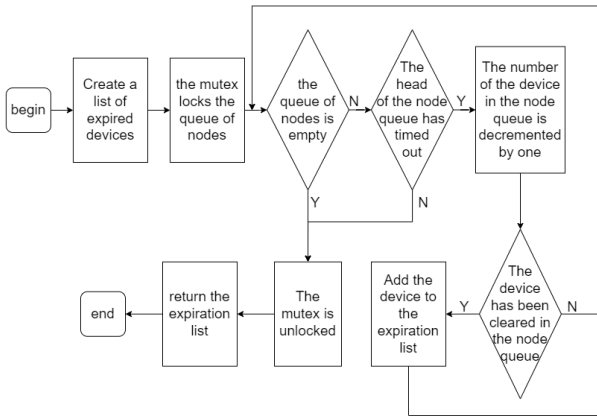


Fig. 4. process in `handleExpired()`.

b) *RPC receiver thread*: The RPC receiver thread is a child thread of the application that handles RPC requests from the Web server. The `RPCServer` instance of the RPC receiving thread is registered by the created `UDPServer` instance of the main thread, and the RPC port is opened. After establishing the RPC connection with the Web server, the `RPCContext` instance is created to save the context of the connection and listen to the control results. The entity relationships of `UDPServer`, `RPCServer` and `RPCContext` are shown in Fig. 5.

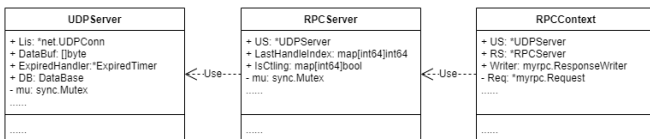


Fig. 5. Entity relationships of `UDPServer`, `RPCServer`, and `RPCContext`.

The lifetime of a created `RPCContext` instance is mainly determined by the following process:

1) Determine whether the control request is valid: First parse the body part of the RPC, determine whether the format of the request is consistent with the control request format, and then determine whether the request has expired according to the `LastIndex` field in the request, if expired, it will directly return a failure;

2) Determine whether the device is online: Since the refresh data of the web side is affected by network fluctuations, the web side may not reflect the online status of the device in time.

Therefore, before controlling, the online status of the device should be checked according to the `UDPServer` pointer when instantiating. If the device has been offline, the control will fail;

3) Determine whether the device is under control: For the safety of device control, this paper designs that only one user can participate in controlling the same device at the same time, so the RPC receiver thread maintains whether each device is in the controlled state. If it is in the controlled state, it will return the control failure, otherwise it will update the control state.

4) Register the pipeline for receiving feedback information, and synchronize the monitoring results: the RPC receiver thread registers a pipeline for receiving control feedback information, and sends the control content to the main thread. The main thread returns the successfully registered pipeline and sends the control content to the device. The RPC receiver thread starts a goroutine containing a timeout to listen to the pipeline. If the goroutine does not receive the result before timeout, the control fails, otherwise it returns the result received by the pipeline.

5) Return RPC packet: The received results are returned to the web server through the RPC connection, and the monitoring pipeline is closed to update the control state.

IV. STORAGE SERVICE DESIGN

The real-time data of the ventilator includes the running device parameters and the body parameters of the patients, etc, and the background data system also has user data, etc. There is a certain correlation between these data, and they need to be persistent and secondary processing, so the database needs to provide the corresponding storage function.

Compared with the traditional relational database MySQL storage scheme, MongoDB as a non-relational database has a more flexible storage type. MySQL usually needs multiple tables to store data due to the limitations of its storage format in systems with complex entity relationships, and in some query scenarios, multiple tables need to be joined to obtain query results. In MongoDB, data is stored in the form of documents, and the format is more free. At the same time, MongoDB also has the advantages of faster read speed, stronger horizontal scaling ability, and support for multiple query methods[11]. Therefore, MongoDB is used as the storage database in this paper.

According to the data characteristics of the ventilator device, this system establishes two sets in the storage service, which are the user information set and the device information set. The user information set is used to store the relevant information of the background user, and the device information set is used to store the running state data of the ventilator.

V. WEB SERVICE PROGRAM DESIGN

The web server program is mainly responsible for providing data to the front-end web page and forwarding control requests to the device server. The system is implemented based on muduo network library[12], and the structure of master-slave Reactor+ thread pool is used to make full use of the performance of multi-core CPU. The log output part is separated from the network connection part, and the

synchronous log system is realized. Parse HTTP requests using a state machine. In addition, the system changes the IO multiplexing mode of muduo from EPOLL-LT to EPOLL-ET, which enhances the performance of the server in handling HTTP short connections. At the same time, cyclic reading (Fig. 6) and cyclic writing are adopted to ensure that read and write errors will not occur due to incomplete read and write buffers in EPOLL-ET mode[13].

readn	
Input: fd - file descriptor; buff - pointer of read buffer; n - number of bytes need read	
Output: readSum - number of bytes actually read	
1	nleft = n, nread = 0, readSum = 0, ptr = buff
2	while nleft > 0 do
3	if (nread = read(fd, ptr, nleft) < 0 then
4	if errno = EINTR then
5	nread = 0
6	else if errno = EAGAIN then
7	return readSum
8	end if
9	else return -1
10	end if
11	else if nread = 0 then
12	break
13	end if
14	readSum += nread, nleft -= nread, ptr += nread
15	end while
16	return readSum

Fig. 6. Pseudocode for cyclic reading.

VI. SYSTEM TEST

The front-end interface was tested through chrome browser, and the user entered the main page through the login interface and clicked on the relevant device to view the device data. The experiment showed that the front-end function of the system was normal and the monitoring function could be completed correctly.

The system test environment is as follows: the test platform is CPU AMD Ryzen 5 3600x, memory dual channel 16GB, and the operating system is Ubuntu20.04. The software development environment is vscode+c++17+Go1.15, the storage service version is MongoDB_4.4.15, and the connection driver versions are mongo-cxx-driver-r3.6.7 and go.mongodb.org/mongo-driver v1.10.3.

A single service stress test was carried out using WebBench[14], 1000 clients were opened, the total test time was 60s, and the log output was turned off. The test results show that the maximum QPS of HTTP short connection is 105485, and the maximum QPS of HTTP long connection is 334687. The results show that the single service Web background in this paper can work normally in a high traffic environment.

The test content includes the number of devices and the heartbeat interval. Considering the influence of network fluctuation in the process of uploading data, the experiment sets a uniform distribution of heartbeat interval from 400ms to 600ms in the last group to simulate the scene of network fluctuation. Since the UDP server is prone to backlog UDP packets in a high traffic environment, the detection of the peak size of the UDP buffer is added to the test. To simplify the test, the test was used to simulate a large number of ventilator conditions.

TABLE I. DEVICE SERVER TEST RESULTS

Number of simulation devices	Interval of heartbeat (ms)	UDP buffer peak size (B)	Peak memory usage (%)	Peak cpu usage (%)
100	500	720	< 0.1	5.3
500	500	720	< 0.1	9.7
500	200	720	< 0.1	16.7
1000	500	720	< 0.1	11.7
1000	200	720	< 0.1	34.3
3000	500	720	< 0.1	33.7
3000	200	720	< 0.1	82.0
3000	400~600(uniform distribution)	720	< 0.1	27.3

Using multiple coroutine clients to carry out simulation experiments on multiple devices, the peak UDP buffer of the device server, the peak server memory and CPU occupation are tested in the case of different device numbers and heartbeat intervals. Considering the influence of network fluctuation in the process of uploading data of actual devices, the experiment set a uniform distribution of heartbeat interval of 400ms to 600ms in the last group to simulate the scene of network fluctuation. Table I shows that in various scenarios, the UDP buffer size of the system remains constant at 720B, the receive buffer does not overflow, and the system can process the device heartbeat data in time. At the same time, the memory occupation of the service is less than 0.1%, which proves that the memory overhead of the service is small and the expired node data is deleted in time. The CPU overhead of the service reaches more than 82% when the number of devices exceeds 3000 and the heartbeat interval is less than 200ms. According to the background data, Golang's frequent garbage collection leads to high CPU occupancy.

VII. CONCLUSION

The system realizes the connection between multiple ventilator devices and the server, and the relevant ventilator devices can be monitored in real time through the Web end. In addition, the system uses UDP protocol to reduce CPU and memory consumption of embedded devices. Meanwhile, the connection reliability is ensured to some extent by the heartbeat and serial number in the system. In terms of background storage, the system uses MongoDB as the database, which has higher read performance and more convenient expansion ability than the traditional MySQL storage scheme. At the same time, the system uses RPC as the communication method between services, which has the characteristics of high security and high efficiency. The backend service adopts dual-service mode, and the Web server is written by C++17, which has the characteristics of high performance and small program size. The server of the device is implemented based on Golang, which not only ensures that the performance is not weaker than the traditional Java SSM and Java SSH frameworks, but also has strong scalability, simple and non-redundant code. In the era of cloud services, Golang's stronger concurrency and network capabilities make it more compatible in the cloud native environment. The stress test results show that the

maximum QPS of HTTP short connection and long connection of Web single service are 105485 and 334687 respectively. The device single service can keep the receive buffer stable and the memory occupation low when the heartbeat interval of 3000 devices is about 500ms.

REFERENCES

- [1] Liu Qian, Huang Li, Li Yuhao, Xu Fen. Analysis of nurses' knowledge, behavior and influencing factors on alarm response fatigue of common monitor [J]. Journal of Southwest Medical University, 201,44(03):256-260.
- [2] Xia Wenjun, Jiang Xuege, Yu Senyang. Application of centralized management mode in ventilator management [J]. Medical Device Information,2005,(4):39-40.
- [3] Xun Yanli, Jiao Ku, Zhang Qinfei. Design and implementation of Smart Home Control System based on Internet of Things [J]. Modern Electronics Technique,2018,(10):74-76.
- [4] Kaiwei Wang,Zhen Li,Xinyu Han,Yunjie Wang.Intelligent control system of chemical process based on Internet of Things[C].International Conference on Internet of Things and Machine Learning (IoTML 2021), 2021, Shanghai, China.
- [5] S. M. Farhad, Matiur Rahman Minar, and Sudipta Majumder. Measurement of Vital Signs with Non-invasive and Wireless Sensing Technologies and Health Monitoring[J]. Journal of Advances in Information Technology.2017,8(3):187-193.
- [6] Shaoyi Guo,Baochen Yang.Research on the application of Internet of Things based on big data cloud computing platform[C]. 2022 International Conference on Cloud Computing, Internet of Things, and Computer Applications, CICA 2022,Luoyang,China.
- [7] Le Thi Thu,Lam Pham Xuan. Towards NoSQL databases: Experiences from actual projects[C]. International Conference on Big Data Analytics and Practices (IBDAP 2022),2022,Bangkok,Thailand.
- [8] James F Kurose,Keith W Ross.Computer Networking:A Top-Down Approach[M]. Beijing: China Machine Press.2013.
- [9] Huang Jiawei, Xia Xing, Feng Zixiao. Building Electric Energy Integrated Control System based on SSM+VUE framework [J]. Computing Technology and Automation,2022,41(01):184-188.
- [10] Chen Shuaihua,Wang Xiaoping.Design of monitoring system for urban low-lying water[C]. International Conference on Mechatronics Technology and Intelligent Manufacturing (ICMTIM 2021), 2021, Hangzhou, China.
- [11] Ji Ying. Research and implementation of Team Collaboration Data Storage Scheme based on MongoDB [D]. University of Chinese Academy of Sciences (School of Engineering Management and Information Technology),2015.
- [12] Chen Shuo. Linux Multithreaded server programming using muduo C++ network library [M]. Beijing: Publishing House of Electronics Industry.2013.
- [13] linyacool.WebServer[EB\OL]. <https://github.com/linyacool/WebServer>.
- [14] EZLippi.WebBench[EB\OL]. <https://github.com/EZLippi/WebBench>.